In [55]:
```
import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense

print("Libraries imported successfully!")
print("TensorFlow Version:", tf.__version__)
```

```
Libraries imported successfully!
TensorFlow Version: 2.22.0-rc0
```

In [56]:
```
data = pd.read_csv(
    r"C:\Users\Adwaith\Downloads\english_hindi.csv",
    encoding="utf-8-sig"
)

print("Dataset loaded successfully!")
print(data.head())

print("\nDataset Shape:", data.shape)
print("\nDataset Columns:", data.columns.tolist())
```

```
Dataset loaded successfully!
          english       hindi
0           hello      नमस्ते
1              hi      नमस्ते
2    good morning    सुप्रभात
3  good afternoon   शुभ दोपहर
4    good evening  शुभ संध्या

Dataset Shape: (93, 2)

Dataset Columns: ['english', 'hindi']
```

In [57]:
```
data = data[["english", "hindi"]].dropna()

data["english"] = data["english"].astype(str).str.strip()
data["hindi"] = data["hindi"].astype(str).str.strip()

data = data[
    (data["english"] != "") &
    (data["hindi"] != "")
].drop_duplicates()

print("Dataset cleaned successfully!")
print("Total sentence pairs:", len(data))
print(data.head())
```

```
Dataset cleaned successfully!
Total sentence pairs: 93
          english       hindi
0           hello      नमस्ते
1              hi      नमस्ते
2    good morning    सुप्रभात
3  good afternoon   शुभ दोपहर
4    good evening  शुभ संध्या
```

In [58]:
```
data["hindi"] = data["hindi"].apply(
    lambda x: "startseq " + x + " endseq"
)

english_sentences = data["english"].tolist()
hindi_sentences = data["hindi"].tolist()

print("Start and end tokens added!")
print("\nEnglish:", english_sentences[0])
print("Hindi:", hindi_sentences[0])
```

```
Start and end tokens added!

English: hello
Hindi: startseq नमस्ते endseq
```

In [59]:
```
translation_dictionary = {}

for english, hindi in zip(
    data["english"],
    data["hindi"]
):

    hindi = hindi.replace("startseq ", "")
    hindi = hindi.replace(" endseq", "")

    translation_dictionary[
        english.lower().strip()
    ] = hindi

print("Translation dictionary created!")
print("Total translations:", len(translation_dictionary))
```

```
Translation dictionary created!
Total translations: 93
```

In [60]:
```
X_train, X_test, y_train, y_test = train_test_split(
    english_sentences,
    hindi_sentences,
    test_size=0.2,
    random_state=42
)

print("Dataset split successfully!")

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Dataset split successfully!
Training samples: 74
Testing samples: 19
```

In [61]:
```
eng_tokenizer = Tokenizer(oov_token="<OOV>")

eng_tokenizer.fit_on_texts(X_train)

encoder_input = eng_tokenizer.texts_to_sequences(X_train)

encoder_test = eng_tokenizer.texts_to_sequences(X_test)

print("English tokenization completed!")

print("English Vocabulary Size:",
      len(eng_tokenizer.word_index) + 1)

print("\nOriginal Sentence:", X_train[0])
print("Tokenized Sentence:", encoder_input[0])
```

```
English tokenization completed!
English Vocabulary Size: 150

Original Sentence: this is my friend
Tokenized Sentence: [7, 2, 5, 34]
```

In [62]:
```
hin_tokenizer = Tokenizer(oov_token="<OOV>")

hin_tokenizer.fit_on_texts(y_train)

decoder_input = hin_tokenizer.texts_to_sequences(y_train)

decoder_test = hin_tokenizer.texts_to_sequences(y_test)

print("Hindi tokenization completed!")

print("Hindi Vocabulary Size:",
      len(hin_tokenizer.word_index) + 1)

print("\nOriginal Sentence:", y_train[0])
print("Tokenized Sentence:", decoder_input[0])
```

```
Hindi tokenization completed!
Hindi Vocabulary Size: 168

Original Sentence: startseq यह मेरा दोस्त है endseq
Tokenized Sentence: [2, 10, 14, 45, 4, 3]
```

In [63]:
```
decoder_target = [
    seq[1:] for seq in decoder_input
]

decoder_target_test = [
    seq[1:] for seq in decoder_test
]

print("Decoder targets created successfully!")

print("Decoder Input:", decoder_input[0])
print("Decoder Target:", decoder_target[0])
```

```
Decoder targets created successfully!
Decoder Input: [2, 10, 14, 45, 4, 3]
Decoder Target: [10, 14, 45, 4, 3]
```

In [64]:
```
max_eng_len = max(len(seq) for seq in encoder_input)

max_hin_len = max(len(seq) for seq in decoder_input)

encoder_input = pad_sequences(
    encoder_input,
    maxlen=max_eng_len,
    padding="post"
)

encoder_test = pad_sequences(
    encoder_test,
    maxlen=max_eng_len,
    padding="post"
)

decoder_input = pad_sequences(
    decoder_input,
    maxlen=max_hin_len,
    padding="post"
)

decoder_test = pad_sequences(
    decoder_test,
    maxlen=max_hin_len,
    padding="post"
)

decoder_target = pad_sequences(
    decoder_target,
    maxlen=max_hin_len,
    padding="post"
)

decoder_target_test = pad_sequences(
    decoder_target_test,
    maxlen=max_hin_len,
    padding="post"
)

print("Padding completed!")

print("Encoder Input Shape:", encoder_input.shape)
print("Decoder Input Shape:", decoder_input.shape)
print("Decoder Target Shape:", decoder_target.shape)
```

```
Padding completed!
Encoder Input Shape: (74, 5)
Decoder Input Shape: (74, 9)
Decoder Target Shape: (74, 9)
```

In [65]:
```
eng_vocab_size = len(eng_tokenizer.word_index) + 1

hin_vocab_size = len(hin_tokenizer.word_index) + 1

print("English Vocabulary Size:", eng_vocab_size)
print("Hindi Vocabulary Size:", hin_vocab_size)

latent_dim = 128
```

```
print("Latent Dimension:", latent_dim)
```

```
English Vocabulary Size: 150
Hindi Vocabulary Size: 168
Latent Dimension: 128
```

In [66]:
```
latent_dim = 128

# Encoder
encoder_inputs = Input(shape=(max_eng_len,))

encoder_embedding = Embedding(
    eng_vocab_size,
    latent_dim,
    mask_zero=True
)(encoder_inputs)

encoder_lstm = LSTM(
    latent_dim,
    return_state=True
)

_, state_h, state_c = encoder_lstm(encoder_embedding)

encoder_states = [state_h, state_c]


# Decoder
decoder_inputs = Input(shape=(max_hin_len,))

decoder_embedding_layer = Embedding(
    hin_vocab_size,
    latent_dim,
    mask_zero=True
)

decoder_embedded = decoder_embedding_layer(decoder_inputs)

decoder_lstm = LSTM(
    latent_dim,
    return_sequences=True,
    return_state=True
)

decoder_outputs, _, _ = decoder_lstm(
    decoder_embedded,
    initial_state=encoder_states
)

decoder_dense = Dense(
    hin_vocab_size,
    activation="softmax"
)

decoder_outputs = decoder_dense(decoder_outputs)


# Complete model
model = Model(
    [encoder_inputs, decoder_inputs],
    decoder_outputs
)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Encoder-Decoder model created successfully!")

model.summary()
```

```
Encoder-Decoder model created successfully!
C:\Users\Adwaith\anaconda3\Lib\site-packages\keras\src\layers\layer.py:1084: User
Warning: Layer 'lstm_6' (of type LSTM) was passed an input with a mask attached t
o it. However, this layer does not support masking and will therefore destroy the
mask information. Downstream layers will not see the mask.
  warnings.warn(
C:\Users\Adwaith\anaconda3\Lib\site-packages\keras\src\layers\layer.py:1084: User
Warning: Layer 'lstm_7' (of type LSTM) was passed an input with a mask attached t
o it. However, this layer does not support masking and will therefore destroy the
mask information. Downstream layers will not see the mask.
  warnings.warn(
```

Model: "functional_11"

Layer (type)	Output Shape	Para
input_layer_18 (InputLayer)	(None, 5)	
input_layer_19 (InputLayer)	(None, 9)	
embedding_6 (Embedding)	(None, 5, 128)	19,
embedding_7 (Embedding)	(None, 9, 128)	21,
lstm_6 (LSTM)	[(None, 128), (None, 128), (None, 128)]	131,
lstm_7 (LSTM)	[(None, 9, 128), (None, 128), (None, 128)]	131,
dense_3 (Dense)	(None, 9, 168)	21,

Total params: 325,544 (1.24 MB)

Trainable params: 325,544 (1.24 MB)

Non-trainable params: 0 (0.00 B)

In [67]:
```
print("\nTraining the Encoder-Decoder model...\n")

history = model.fit(
    [encoder_input, decoder_input],
    decoder_target,
    validation_split=0.1,
    epochs=30,
    batch_size=4,
    verbose=1
)
```

```
print("\nModel training completed!")
```

```
Training the Encoder-Decoder model...

Epoch 1/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 5s 134ms/step - accuracy: 0.4529 - loss: 4.7816 - val_
accuracy: 0.5000 - val_loss: 3.5033
Epoch 2/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.4899 - loss: 2.7937 - val_a
ccuracy: 0.5000 - val_loss: 2.3751
Epoch 3/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4899 - loss: 2.3607 - val_a
ccuracy: 0.5000 - val_loss: 2.2914
Epoch 4/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.4899 - loss: 2.2290 - val_a
ccuracy: 0.5417 - val_loss: 2.2436
Epoch 5/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5572 - loss: 2.1298 - val_a
ccuracy: 0.5833 - val_loss: 2.2037
Epoch 6/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5892 - loss: 2.0658 - val_a
ccuracy: 0.5972 - val_loss: 2.2083
Epoch 7/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.5943 - loss: 2.0360 - val_a
ccuracy: 0.5972 - val_loss: 2.1921
Epoch 8/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5976 - loss: 2.0117 - val_a
ccuracy: 0.6111 - val_loss: 2.1830
Epoch 9/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5993 - loss: 2.0433 - val_a
ccuracy: 0.6111 - val_loss: 2.2183
Epoch 10/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6061 - loss: 1.9741 - val_a
ccuracy: 0.6111 - val_loss: 2.2126
Epoch 11/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6111 - loss: 1.9251 - val_a
ccuracy: 0.6111 - val_loss: 2.1882
Epoch 12/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6145 - loss: 1.8886 - val_a
ccuracy: 0.6111 - val_loss: 2.2387
Epoch 13/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6178 - loss: 1.8540 - val_a
ccuracy: 0.6111 - val_loss: 2.2280
Epoch 14/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6246 - loss: 1.8143 - val_a
ccuracy: 0.6111 - val_loss: 2.2117
Epoch 15/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6246 - loss: 1.7938 - val_a
ccuracy: 0.5833 - val_loss: 2.2287
Epoch 16/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6347 - loss: 1.8281 - val_a
ccuracy: 0.6111 - val_loss: 2.3753
Epoch 17/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6481 - loss: 1.7418 - val_a
ccuracy: 0.6111 - val_loss: 2.2772
Epoch 18/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6465 - loss: 1.6400 - val_a
ccuracy: 0.6111 - val_loss: 2.2841
Epoch 19/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6498 - loss: 1.5640 - val_a
ccuracy: 0.6111 - val_loss: 2.2682
Epoch 20/30
```

```
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6599 - loss: 1.4955 - val_a
ccuracy: 0.6389 - val_loss: 2.2543
Epoch 21/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.6734 - loss: 1.4265 - val_ac
curacy: 0.6389 - val_loss: 2.2510
Epoch 22/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6734 - loss: 1.3560 - val_a
ccuracy: 0.6389 - val_loss: 2.2586
Epoch 23/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.6835 - loss: 1.2817 - val_a
ccuracy: 0.6528 - val_loss: 2.2708
Epoch 24/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.6987 - loss: 1.2025 - val_ac
curacy: 0.6667 - val_loss: 2.2941
Epoch 25/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7104 - loss: 1.1267 - val_a
ccuracy: 0.6667 - val_loss: 2.3135
Epoch 26/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7239 - loss: 1.0553 - val_a
ccuracy: 0.6806 - val_loss: 2.3131
Epoch 27/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7222 - loss: 1.0085 - val_a
ccuracy: 0.6806 - val_loss: 2.5063
Epoch 28/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7492 - loss: 0.9726 - val_a
ccuracy: 0.6944 - val_loss: 2.4844
Epoch 29/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7559 - loss: 0.8964 - val_a
ccuracy: 0.6806 - val_loss: 2.4319
Epoch 30/30
17/17 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7862 - loss: 0.8176 - val_a
ccuracy: 0.6944 - val_loss: 2.5070

Model training completed!
```

In [68]:
```python
loss, accuracy = model.evaluate(
    [encoder_test, decoder_test],
    decoder_target_test,
    verbose=0
)

print("\nModel Evaluation Results")

print("Test Loss:", round(float(loss), 4))

print(
    "Test Token Accuracy:",
    round(float(accuracy) * 100, 2),
    "%"
)
```

```
Model Evaluation Results
Test Loss: 2.934
Test Token Accuracy: 64.91 %
```

In [69]:
```python
# Encoder inference model
encoder_model = Model(
    encoder_inputs,
    encoder_states
)
```

```python
# Decoder inference inputs
state_input_h = Input(shape=(latent_dim,))
state_input_c = Input(shape=(latent_dim,))

decoder_states_inputs = [
    state_input_h,
    state_input_c
]

decoder_single_input = Input(shape=(1,))

decoder_single_embedded = decoder_embedding_layer(
    decoder_single_input
)

decoder_single_output, h_new, c_new = decoder_lstm(
    decoder_single_embedded,
    initial_state=decoder_states_inputs
)

decoder_single_output = decoder_dense(
    decoder_single_output
)

# Decoder inference model
decoder_model = Model(
    [decoder_single_input] + decoder_states_inputs,
    [decoder_single_output, h_new, c_new]
)

print("Encoder and Decoder inference models created!")
```

```
Encoder and Decoder inference models created!
```

In [71]:
```python
# Reverse Hindi vocabulary
reverse_hin_index = {
    index: word
    for word, index in hin_tokenizer.word_index.items()
}

start_token = hin_tokenizer.word_index["startseq"]

end_token = hin_tokenizer.word_index["endseq"]


# Translation function
def translate_sentence(sentence):

    sentence = sentence.lower().strip()

    # First check the dataset dictionary
    if sentence in translation_dictionary:

        return translation_dictionary[sentence]

    # Otherwise use the trained LSTM model
    sequence = eng_tokenizer.texts_to_sequences([sentence])

    sequence = pad_sequences(
```

```
        sequence,
        maxlen=max_eng_len,
        padding="post"
    )

    states = encoder_model.predict(
        sequence,
        verbose=0
    )

    target_seq = np.array([[start_token]])

    translated_words = []

    for _ in range(max_hin_len):

        output, h, c = decoder_model.predict(
            [target_seq] + states,
            verbose=0
        )

        predicted_id = int(
            np.argmax(output[0, -1, :])
        )

        if predicted_id == 0 or predicted_id == end_token:
            break

        word = reverse_hin_index.get(
            predicted_id, ""
        )

        if word and word not in ("startseq", "<OOV>"):
            translated_words.append(word)

        target_seq = np.array([[predicted_id]])

        states = [h, c]

    return (
        " ".join(translated_words)
        if translated_words
        else "Translation could not be generated."
    )
```

In [73]:

```
print("\n==================================")
print(" ENGLISH TO HINDI TRANSLATOR")
print("==================================")

print("Enter 'exit' to stop the program.")

while True:

    user_sentence = input(
        "\nEnter an English sentence: "
    ).strip()

    if user_sentence.lower() == "exit":

        print("Translation program stopped.")
        break

    if not user_sentence:

        print("Please enter a sentence.")
        continue

    hindi_translation = translate_sentence(
        user_sentence
    )

    print("Hindi Translation:", hindi_translation)


# Save trained model
model.save(
    r"C:\Users\Adwaith\Downloads\english_hindi_model.keras"
)

print("Model saved successfully!")
```

```
==================================
 ENGLISH TO HINDI TRANSLATOR
==================================
Enter 'exit' to stop the program.
Hindi Translation: नमस्ते
Hindi Translation: नमस्ते
Hindi Translation: सुप्रभात
Hindi Translation: शुभ दोपहर
Hindi Translation: आप कैसे हैं
Hindi Translation: धन्यवाद
Hindi Translation: मैं ठीक हूँ
Hindi Translation: अलविदा
Translation program stopped.
Model saved successfully!
```

In [ ]: